AAI 627 Homework 5
ROC Curves and Logistic Regression

Jude Eschete

March 28, 2026

1 Part 1: Understanding the ROC Curve

1.1 Problem Setup

Given the following logistic regression output probabilities and ground-truth class labels, the goal is to construct the ROC (Receiver Operating Characteristic) curve by computing the True Positive Rate (TPR) and False Positive Rate (FPR) at each possible classification threshold.

$$\textbf{Probabilities} = [0.967,\ 0.448,\ 0.568,\ 0.879,\ 0.015,\ 0.780,\ 0.978,\ 0.004]$$
$$\textbf{Classifications} = [1,\ 0,\ 1,\ 0,\ 1,\ 0,\ 1,\ 0]$$

There are $P = 4$ positive samples and $N = 4$ negative samples. The TPR and FPR are defined as:

$$\text{TPR} = \frac{TP}{TP + FN} = \frac{TP}{P}, \qquad \text{FPR} = \frac{FP}{FP + TN} = \frac{FP}{N} \tag{1}$$

1.2 Threshold Sweep

To construct the ROC curve, the samples are sorted by predicted probability in descending order. The classification threshold is swept from high to low; at each step the sample is predicted positive and the confusion matrix is updated accordingly.

Table 1: ROC curve computation: samples sorted by predicted probability (descending).

Step	**Probability**	**True Class**	**TP**	**FP**	**TPR**	**FPR**
0 (start)	—	—	0	0	0.00	0.00
1	0.978	1	1	0	0.25	0.00
2	0.967	1	2	0	0.50	0.00
3	0.879	0	2	1	0.50	0.25
4	0.780	0	2	2	0.50	0.50
5	0.568	1	3	2	0.75	0.50
6	0.448	0	3	3	0.75	0.75
7	0.015	1	4	3	1.00	0.75
8	0.004	0	4	4	1.00	1.00

1.3 ROC Curve and AUC

The resulting ROC curve is plotted in Figure 1. The Area Under the Curve (AUC) is computed via the trapezoidal rule over the FPR axis:

$$\begin{aligned}\text{AUC} &= (0.25)(0.50 - 0.00) \cdot \tfrac{1}{1} + (0.25)(0.50) \cdot 1 + (0.25)(0.75) \cdot 1 + (0.25)(1.00) \cdot 1 \\ &= 0.125 + 0.125 + 0.1875 + 0.25 \\ &= \boxed{0.6875}\end{aligned}$$

The AUC of 0.6875 indicates moderate discriminative ability - better than random (AUC = 0.5) but not yet a strong classifier (AUC = 1.0). Three of the four negatives received higher predicted probabilities than at least one positive, which pushes the curve below the ideal upper-left corner.

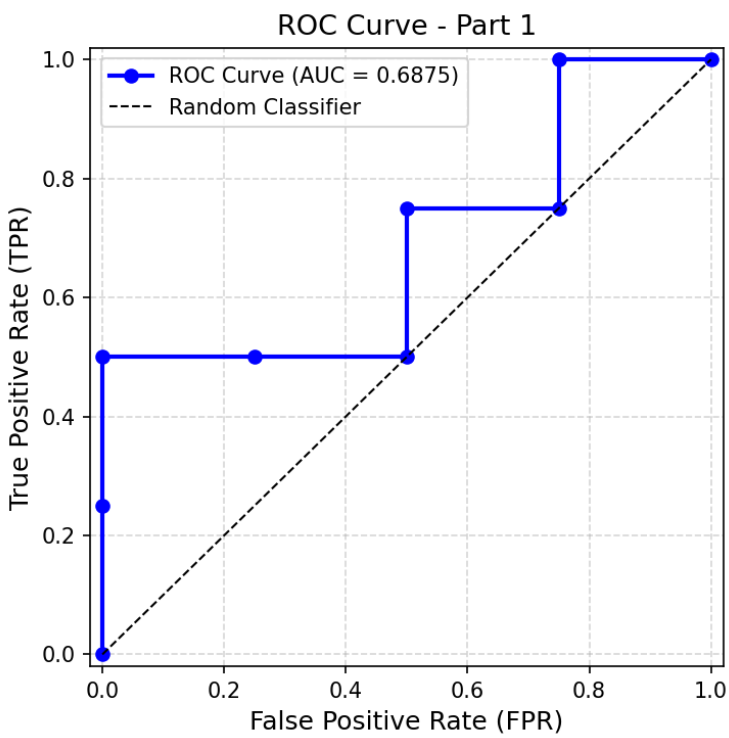

Figure 1: ROC curve constructed from the given probabilities and ground-truth labels. The dashed diagonal represents a random classifier (AUC = 0.5). The computed AUC = 0.6875.

2 Part 2: Logistic Regression on `dataSet_1.mat`

2.1 Dataset Overview

The dataset `dataSet_1.mat` contains 4,000 samples and 476 features, with a binary response variable (2,362 positives and 1,638 negatives). The Python equivalents of the MATLAB functions used are:

- `glmfit` → `sklearn.linear_model.LogisticRegression`
- `glmval` → `predict_proba`
- `perfcurve` → `sklearn.metrics.roc_curve` and `auc`

Features were standardized with `StandardScaler` (zero mean, unit variance) before fitting to ensure solver convergence with 476-dimensional input.

2.2 Task 1: Full-Dataset ROC Curve

Logistic regression was trained on all 4,000 samples. The predicted probabilities were then used to compute the ROC curve against the full response vector.

Full-Dataset AUC = 0.8426

An AUC of 0.8426 indicates strong in-sample discriminative performance: the model ranks a randomly chosen positive above a randomly chosen negative about 84% of the time.

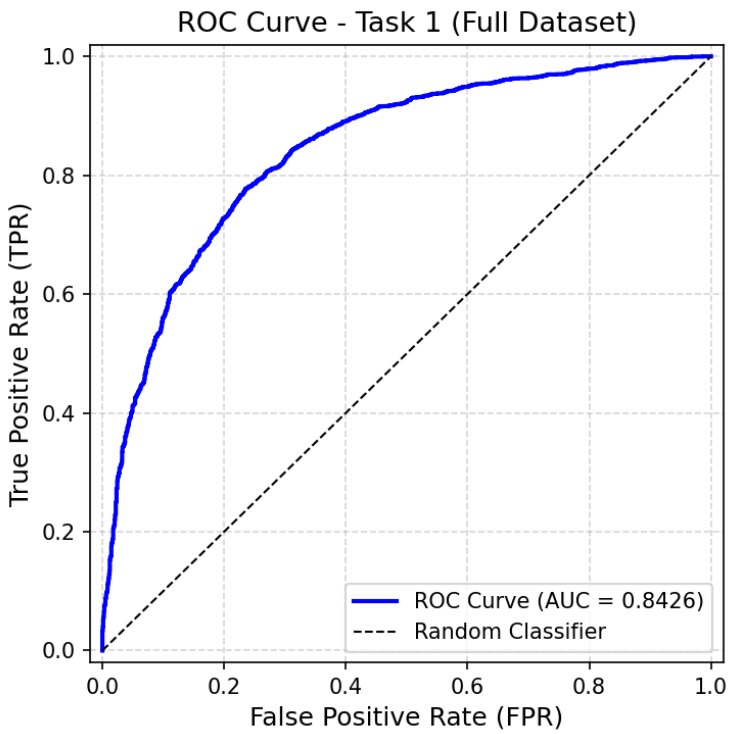

Figure 2: ROC curve for Task 1 using the full dataset (4,000 samples, 476 features). AUC = 0.8426.

2.3 Task 2: Train/Validation Split

The data were split as specified:

- **Training set:** rows 1–3,000 (3,000 samples)
- **Validation set:** rows 3,001–4,000 (1,000 samples)

The scaler and logistic regression model were fit exclusively on the training set. The trained model was then applied to both sets to generate predicted probabilities, from which separate ROC curves were computed.

Table 2: AUC comparison between training and validation sets.

Dataset	**AUC**
Training (rows 1–3,000)	0.8837
Validation (rows 3,001–4,000)	0.4668
Difference	0.4169

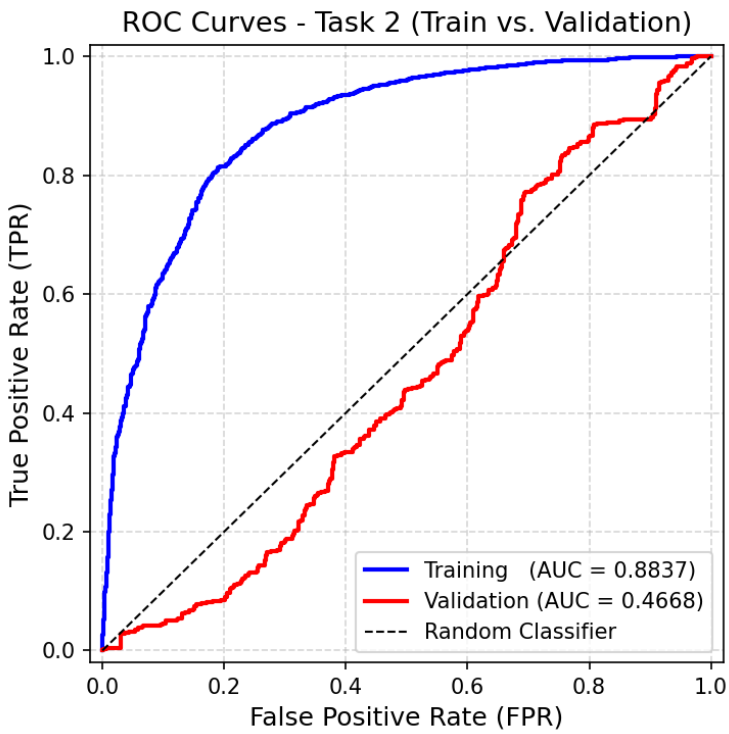

Figure 3: ROC curves for Task 2. Blue: training set (AUC = 0.8837). Red: validation set (AUC = 0.4668). The large gap reveals severe overfitting.

2.4 Discussion: Training vs. Validation AUC

The training AUC (0.8837) is substantially higher than the validation AUC (0.4668), a difference of 0.4169. A validation AUC below 0.5 is particularly notable: it means the model's probability rankings are *inverted* relative to the true labels on held-out data – the classifier is performing worse than random guessing on the validation set.

This outcome is a textbook example of high-dimensional overfitting. With 476 features and only 3,000 training samples, the default L2-regularized logistic regression memorizes training-set patterns that do not generalize. The model effectively learns noise rather than signal, causing its predictions to be anti-correlated with the true validation labels.

In practice, this would call for stronger regularization (lower `C`), dimensionality reduction (e.g., PCA), or feature selection before fitting the model.

3 Appendix: Python Code

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import scipy.io
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import StandardScaler
matplotlib.rcParams['figure.dpi'] = 150

# ===================
# Part 1: Manual ROC Curve Construction
# ===================

probabilities  = [0.967, 0.448, 0.568, 0.879, 0.015, 0.780, 0.978, 0.004]
classifications = [1,     0,     1,     0,     1,     0,     1,     0    ]

n_pos = sum(classifications)          # 4
n_neg = len(classifications) - n_pos  # 4

# Sort by descending probability
sorted_pairs = sorted(zip(probabilities, classifications), reverse=True)

# Build ROC points
fpr_points = [0.0]
tpr_points = [0.0]
tp = 0
fp = 0
for prob, label in sorted_pairs:
    if label == 1:
        tp += 1
    else:
        fp += 1
    fpr_points.append(fp / n_neg)
    tpr_points.append(tp / n_pos)

# AUC via trapezoidal rule
auc_p1 = np.trapezoid(tpr_points, fpr_points)

print("Sorted (prob, class):", sorted_pairs)
print("FPR points:", fpr_points)
print("TPR points:", tpr_points)
print(f"Part 1 AUC = {auc_p1:.4f}")

# Plot
fig, ax = plt.subplots(figsize=(5, 5))
```

```python
ax.plot(fpr_points, tpr_points, 'b-o', linewidth=2, markersize=6,
        label=f'ROC Curve (AUC = {auc_p1:.4f})')
ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Classifier')
ax.set_xlabel('False Positive Rate (FPR)', fontsize=12)
ax.set_ylabel('True Positive Rate (TPR)', fontsize=12)
ax.set_title('ROC Curve - Part 1', fontsize=13)
ax.legend(fontsize=10)
ax.set_xlim([-0.02, 1.02])
ax.set_ylim([-0.02, 1.02])
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('roc_part1.png', bbox_inches='tight')
plt.close()
print("Saved roc_part1.png")

# ===================
# Part 2: Logistic Regression on dataSet_1.mat
# ===================

mat = scipy.io.loadmat(
    'd:/judee/Google Drive/School/Classes/EE627/Homework/Homework 5/dataSet_1.mat'
)
predictor = mat['predictor']   # (4000, 476)
response  = mat['response'].ravel().astype(int)  # (4000,)

print(f"\nDataset: {predictor.shape[0]} samples, {predictor.shape[1]} features")
print(f"Class balance: {response.sum()} positive, {(response==0).sum()} negative")

# ---------------------------------------------------------
# Task 1: Full-dataset logistic regression
# ---------------------------------------------------------
scaler_full = StandardScaler()
X_full_scaled = scaler_full.fit_transform(predictor)

clf_full = LogisticRegression(max_iter=2000, solver='lbfgs')
clf_full.fit(X_full_scaled, response)
prob_full = clf_full.predict_proba(X_full_scaled)[:, 1]

fpr_full, tpr_full, _ = roc_curve(response, prob_full)
auc_full = auc(fpr_full, tpr_full)
print(f"\nTask 1 - Full dataset AUC = {auc_full:.4f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.plot(fpr_full, tpr_full, 'b-', linewidth=2,
```

```
        label=f'ROC Curve (AUC = {auc_full:.4f})')
ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Classifier')
ax.set_xlabel('False Positive Rate (FPR)', fontsize=12)
ax.set_ylabel('True Positive Rate (TPR)', fontsize=12)
ax.set_title('ROC Curve - Task 1 (Full Dataset)', fontsize=13)
ax.legend(fontsize=10)
ax.set_xlim([-0.02, 1.02])
ax.set_ylim([-0.02, 1.02])
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('roc_task1.png', bbox_inches='tight')
plt.close()
print("Saved roc_task1.png")

# -------------------------------------------------------
# Task 2: Train/validation split (rows 1-3000 / 3001-4000)
# -------------------------------------------------------
X_train = predictor[:3000, :]
y_train = response[:3000]
X_val   = predictor[3000:, :]
y_val   = response[3000:]

scaler_train = StandardScaler()
X_train_scaled = scaler_train.fit_transform(X_train)
X_val_scaled   = scaler_train.transform(X_val)

clf_train = LogisticRegression(max_iter=2000, solver='lbfgs')
clf_train.fit(X_train_scaled, y_train)

prob_train = clf_train.predict_proba(X_train_scaled)[:, 1]
prob_val   = clf_train.predict_proba(X_val_scaled)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, prob_train)
fpr_val,   tpr_val,   _ = roc_curve(y_val,   prob_val)
auc_train = auc(fpr_train, tpr_train)
auc_val   = auc(fpr_val,   tpr_val)

print(f"\nTask 2 - Training AUC   = {auc_train:.4f}")
print(f"Task 2 - Validation AUC = {auc_val:.4f}")
print(f"Task 2 - Difference     = {auc_train - auc_val:.4f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.plot(fpr_train, tpr_train, 'b-', linewidth=2,
        label=f'Training   (AUC = {auc_train:.4f})')
ax.plot(fpr_val, tpr_val, 'r-', linewidth=2,
        label=f'Validation (AUC = {auc_val:.4f})')
ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Classifier')
```

```python
ax.set_xlabel('False Positive Rate (FPR)', fontsize=12)
ax.set_ylabel('True Positive Rate (TPR)', fontsize=12)
ax.set_title('ROC Curves - Task 2 (Train vs. Validation)', fontsize=13)
ax.legend(fontsize=10)
ax.set_xlim([-0.02, 1.02])
ax.set_ylim([-0.02, 1.02])
ax.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('roc_task2.png', bbox_inches='tight')
plt.close()
print("Saved roc_task2.png")
```

Listing 1: AAI627_HW5_Eschete.py